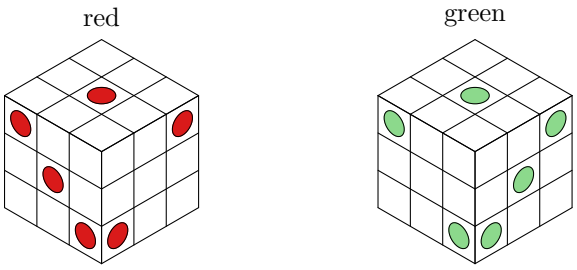
September 5, 2026 at 21:04

1. Introduction. Exercise 7.2.2.1–337 asks for a puzzle of Angus Lavery's: nine bent tricubes, their square faces blank or bearing a red or a green spot, that assemble into a $3 \times 3 \times 3$ cube in two ways—once showing a left-handed die in red with no green in sight, and once showing a right-handed die in green with no red.

Answer 337 works it out and reports a string of numbers along the way: nine bent tricubes pack the cube in 5328 ways, falling into 111 classes of 48; each piece has fourteen faces of which two can never show; an assembly fixes from 2 to 7 of the twelve that can, so 21 faces come out red, 33 blank and 54 free; 371 of the 5328 red solutions can be rearranged into green ones, one of them in 6048 different ways; and 52 red-and-green combinations leave 18 faces unspecified. This program checks all of that, and every number comes out.

The two spot patterns are the point of the puzzle, so here they are, drawn from the corner where the 1, the 2 and the 3 meet—which is the corner that decides a die's handedness.




```
package main
import (
    "flag"
    "fmt"
    "sort"
    "strings"
    cells "github.com/sjnam/dancing-cells"
)
⟨ Types 4 ⟩
⟨ Functions 5 ⟩
func main() {
    ⟨ Read the command line 2 ⟩
    ⟨ Do what the mode asks 3 ⟩
}
```

2. ⟨ Read the command line 2 ⟩ ≡

```
mode := flag.String("mode", "all", "pack␣die␣faces␣twice␣or␣all")
lo := flag.Int("lo", 0, "first␣red␣solution␣to␣try")
hi := flag.Int("hi", 5328, "one␣past␣the␣last")
flag.Parse()
```

This code is used in section 1.

3. ⟨ Do what the mode asks 3 ⟩ ≡

```
if *mode ≡ "pack" ∨ *mode ≡ "all" {
    ⟨ Count the packings and their classes 24 ⟩
}
if *mode ≡ "die" ∨ *mode ≡ "all" {
    ⟨ Check the two spot patterns 37 ⟩
}
if *mode ≡ "faces" ∨ *mode ≡ "all" {
    ⟨ Count what an assembly fixes 43 ⟩
}
if *mode ≡ "twice" ∨ *mode ≡ "all" {
    ⟨ Rearrange every red solution 51 ⟩
}
```

This code is used in section 1.

4. The cube and its symmetries. Answer 337 uses the even/odd coordinates of exercise 145: the cubie in position (i, j, k) sits at $(2i+1, 2j+1, 2k+1)$, so that each of its faces has one coordinate in $\{0, 6\}$ and two in $\{1, 3, 5\}$ and can be named by three digits.

⟨ Types 4 ⟩ ≡

```
type vec struct { x, y, z int }
```

This code is also defined in sections 7, 12, 14, 18, 38, 41.

This code is used in section 1.

5. ⟨ Functions 5 ⟩ ≡

```
func add(a, b vec) vec { return vec{a.x + b.x, a.y + b.y, a.z + b.z} }
func outside(c vec) bool {
    return c.x < 0 ∨ c.x > 2 ∨ c.y < 0 ∨ c.y > 2 ∨ c.z < 0 ∨ c.z > 2
}
func faceName(c, d vec) string {
    return fmt.Sprintf("%d%d%d", 2 * c.x + 1 + d.x, 2 * c.y + 1 + d.y, 2 * c.z + 1 + d.z)
}
```

This code is also defined in sections 6, 8, 9, 11, 13, 15, 17, 19, 21, 22, 23, 25, 26, 28, 29, 32, 33, 36, 39, 40, 42, 46, 47, 50.

This code is used in section 1.

6. ⟨ Functions 5 ⟩ +≡

```
var dirs = []vec{{1, 0, 0}, {−1, 0, 0}, {0, 1, 0}, {0, −1, 0}, {0, 0, 1}, {0, 0, −1}}
```

7. The symmetries of the cube are the signed permutation matrices: 48 of them, 24 with determinant 1. The proper ones are the rotations, which are all a physical piece may be given; the whole 48 are wanted when packings are to be counted up to reflection as well.

⟨ Types 4 ⟩ +≡

```
type mat [3]vec
```

8. ⟨ Functions 5 ⟩ +≡

```
func mul(m mat, v vec) vec {
    return vec{
        m[0].x * v.x + m[0].y * v.y + m[0].z * v.z,
        m[1].x * v.x + m[1].y * v.y + m[1].z * v.z,
        m[2].x * v.x + m[2].y * v.y + m[2].z * v.z,
    }
}
func det3(m mat) int {
    return m[0].x * (m[1].y * m[2].z − m[1].z * m[2].y) −
        m[0].y * (m[1].x * m[2].z − m[1].z * m[2].x) +
        m[0].z * (m[1].x * m[2].y − m[1].y * m[2].x)
}
```

9. ⟨ Functions 5 ⟩ +≡

```
func allMats(proper bool) []mat {
    var out []mat
    perms := [][3]int{{0,1,2},{0,2,1},{1,0,2},{1,2,0},{2,0,1},{2,1,0}}
    for _, p := range perms {
        for s := 0; s < 8; s++ {
            ⟨ Build the signed permutation and keep it if wanted 10 ⟩
        }
    }
    return out
}
```

10. ⟨ Build the signed permutation and keep it if wanted 10 ⟩ ≡

```
var m mat
for i := 0; i < 3; i++ {
    sgn := 1
    if s >> i & 1 == 1 {
        sgn = -1
    }
    switch p[i] {
    case 0:
        m[i] = vec{sgn, 0, 0}
    case 1:
        m[i] = vec{0, sgn, 0}
    default:
        m[i] = vec{0, 0, sgn}
    }
}
if !proper || det3(m) == 1 {
    out = append(out, m)
}
```

This code is used in section 9.

11. The bent tricube. Three cubies in an L. Of its eighteen cubie faces four are glued away, leaving the fourteen the answer counts; and two of those look into the notch of its $2\times2\times1$ block, so they are covered by another piece in any packing and can never show.

⟨ Functions 5 ⟩ +≡

```
var pieceCells = []vec{{0, 0, 0}, {1, 0, 0}, {1, 1, 0}}
```

12. ⟨ Types 4 ⟩ +≡

```
type face struct {
    c, d vec
    inner bool
}
```

13. ⟨ Functions 5 ⟩ +≡

```
func pieceFaces() []face {
    in := map[vec]bool{}
    for _, c := range pieceCells {
        in[c] = true
    }
    notch := vec{0, 1, 0}
    var out []face
    for _, c := range pieceCells {
        for _, d := range dirs {
            if in[add(c, d)] {
                continue
            }
            out = append(out, face{c, d, add(c, d) ≡ notch})
        }
    }
    return out
}
```

14. A placement puts the piece somewhere in the cube in one of its 24 rotations, and remembers where each of its fourteen faces went. The bent tricube has a symmetry of its own—a half-turn about the diagonal that swaps its two arms—so two rotations fill the same cells; but they carry the faces to different places, and a piece with spots on it can tell them apart. There turn out to be 288 of them, filling 144 distinct sets of cells.

⟨ Types 4 ⟩ +≡

```
type place struct {
    cells [3]vec
    at, dir []vec
    key string
}
```

15. ⟨ Functions 5 ⟩ +≡

```
func placements() []place {
    fs := pieceFaces()
    seen := map[string]bool{}
    var out []place
    for _, m := range allMats(true) {
        for x := -2; x <= 2; x++ {
            for y := -2; y <= 2; y++ {
                for z := -2; z <= 2; z++ {
                    ⟨ Place the piece at (x, y, z) under m 16 ⟩
                }
            }
        }
    }
    return out
}
```

16. ⟨ Place the piece at (x, y, z) under m 16 ⟩ ≡

```
t := vec{x, y, z}
var p place
ok := true
for i, c := range pieceCells {
    q := add(mul(m, c), t)
    if outside(q) {
        ok = false
        break
    }
    p.cells[i] = q
}
if ¬ok {
    continue
}
for _, f := range fs {
    p.at = append(p.at, add(mul(m, f.c), t))
    p.dir = append(p.dir, mul(m, f.d))
}
p.key = placeKey(p)
if ¬seen[p.key] {
    seen[p.key] = true
    out = append(out, p)
}
```

This code is used in section 15.

17. ⟨ Functions 5 ⟩ +≡

```
func placeKey(p place) string {
    var b strings.Builder
    for i := range p.at {
        fmt.Fprintf(&b, "%v%v;", p.at[i], p.dir[i])
    }
    return b.String()
}
func cellKey(cs [3]vec) string {
    var s []string
    for _, c := range cs {
        s = append(s, fmt.Sprintf("%d%d%d", c.x, c.y, c.z))
    }
    sort.Strings(s)
    return strings.Join(s, "")
}
```

18. Packing the cube. The pieces are identical until they are painted, so the packing problem has only the 27 cells as items and one option per set of cells a piece can fill. Each solution is then a partition of the cube, counted once.

⟨ Types 4 ⟩ +≡

```
type packing []string
```

19. ⟨ Functions 5 ⟩ +≡

```
func packProblem() string {
    byCell := map[string]bool{}
    for _, p := range placements() {
        byCell[cellKey(p.cells)] = true
    }
    var keys []string
    for k := range byCell {
        keys = append(keys, k)
    }
    sort.Strings(keys)
    var b strings.Builder
    ⟨ Write the cell items 20 ⟩
    for _, k := range keys {
        for i := 0; i < 9; i += 3 {
            fmt.Fprintf(&b, "c%s␣", k[i:i+3])
        }
        b.WriteString("\n")
    }
    return b.String()
}
```

20. ⟨ Write the cell items 20 ⟩ ≡

```
for x := 0; x < 3; x++ {
    for y := 0; y < 3; y++ {
        for z := 0; z < 3; z++ {
            fmt.Fprintf(&b, "c%d%d%d␣", x, y, z)
        }
    }
}
b.WriteString("\n")
```

This code is used in sections 19, 47.

21. ⟨ Functions 5 ⟩ +≡

```
func allPackings() []packing {
    var out []packing
    xc := cells.NewXCC()
    for sol := range xc.Dance(strings.NewReader(packProblem())).Solutions {
        var p packing
        for _, o := range sol {
            var cs []string
            for _, t := range o {
                cs = append(cs, t[1:])
            }
            sort.Strings(cs)
            p = append(p, strings.Join(cs, ""))
        }
        sort.Strings(p)
        out = append(out, p)
    }
    return out
}
```

22. Two packings are equivalent when a symmetry of the cube—about its centre, so the coordinates are shifted by one before the matrix is applied—carries one to the other.

⟨ Functions 5 ⟩ +≡

```
func (p packing) image(m mat) packing {
    var q packing
    for _, t := range p {
        var cs []string
        for i := 0; i < 9; i += 3 {
            c := vec{int(t[i] - '0'), int(t[i + 1] - '0'), int(t[i + 2] - '0')}
            d := add(mul(m, vec{c.x - 1, c.y - 1, c.z - 1}), vec{1, 1, 1})
            cs = append(cs, fmt.Sprintf("%d%d%d", d.x, d.y, d.z))
        }
        sort.Strings(cs)
        q = append(q, strings.Join(cs, ""))
    }
    sort.Strings(q)
    return q
}
func (p packing) key() string { return strings.Join(p, "|") }
```

23. ⟨ Functions 5 ⟩ +≡

```
func (p packing) canon()(string, int) {
    best, fix := "", 0
    for _, m := range allMats(false) {
        k := p.image(m).key()
        if best ≡ "" ∨ k < best {
            best = k
        }
        if k ≡ p.key() {
            fix++
        }
    }
    return best, fix
}
```

24. ⟨ Count the packings and their classes 24 ⟩ ≡

```
ps := allPackings()
cl := map[string]int{}
stab := map[int]int{}
for _, q := range ps {
    k, fix := q.canon()
    cl[k]++
    stab[fix]++
}
sizes := map[int]int{}
for _, v := range cl {
    sizes[v]++
}
fmt.Printf("%d␣packings,␣%d␣classes,␣sizes␣%v,␣stabilizer␣orders␣%v\n",
    len(ps), len(cl), sizes, stab)
```

This code is used in section 3.

25. The two dice. Answer 337 gives the red spots as 21 faces and says the green ones are the same except that 303 replaces 033 and 633 replaces 363. Before anything is built on them it is worth asking whether they really are dice: the usual pip patterns, opposite sides adding to seven, and one of each handedness.

⟨ Functions 5 ⟩ +≡

```
var redSpots = strings.Fields(`330 105 501 015 033 051 611 615 651 655
 161 165 363 561 565 116 136 156 516 536 556`)
func greenSpots() []string {
    var out []string
    for _, s := range redSpots {
        switch s {
        case "033":
            out = append(out, "303")
        case "363":
            out = append(out, "633")
        default:
            out = append(out, s)
        }
    }
    return out
}
```

26. A face name has one coordinate in $\{0, 6\}$, which says which side of the cube it is on; the other two place the pip within that side.

⟨ Functions 5 ⟩ +≡

```
func sideOf(s string)(vec, [2]int) {
    c := [3]int{int(s[0] - '0'), int(s[1] - '0'), int(s[2] - '0')}
    for i := 0; i < 3; i++ {
        if c[i] ≡ 0 ∨ c[i] ≡ 6 {
            ⟨ Return the outward normal and the pip's place 27 ⟩
        }
    }
    panic("not a face of the cube")
}
```

27. ⟨ Return the outward normal and the pip's place 27 ⟩ ≡

```
d := -1
if c[i] ≡ 6 {
    d = 1
}
n := [3]vec{{d, 0, 0}, {0, d, 0}, {0, 0, d}}[i]
var rest [2]int
k := 0
for j := 0; j < 3; j++ {
    if j ≠ i {
        rest[k] = c[j]
        k++
    }
}
return n, rest
```

This code is used in section 26.

28. The handedness is the one thing the coordinates do not settle by themselves. Taking the outward normals of the 1, the 2 and the 3 as the rows of a matrix, a positive determinant means those three sides run counterclockwise about their common corner, which is what is usually called right-handed—but only if (x, y, z) is read as a right-handed frame, and nothing says that it is. The answer's own picture does: it draws the red die with 1 on top, 3 at the front left and 2 at the right, so that 1, 2, 3 run clockwise, and the red die is left-handed as the exercise says. So the frame of the coordinates is meant to be left-handed, and this program reports the determinant and lets the reader supply the convention.

```
⟨ Functions 5 ⟩ +≡
    func checkDie(spots []string, name string) {
        sides := map[vec][][2]int{}
        for _, s := range spots {
            n, r := sideOf(s)
            sides[n] = append(sides[n], r)
        }
        fmt.Printf("%s: %d spots on %d sides\n", name, len(spots), len(sides))
        ⟨ Report each side's pattern 30 ⟩
        ⟨ Report the opposite sums and the handedness 31 ⟩
    }
```

29.

```
⟨ Functions 5 ⟩ +≡
    var pipPattern = map[int]string{
        1: "33",
        2: "15 51",
        3: "15 33 51",
        4: "11 15 51 55",
        5: "11 15 33 51 55",
        6: "11 13 15 51 53 55",
    }
```

30.

```
⟨ Report each side's pattern 30 ⟩ ≡
    for _, n := range dirs {
        var ss []string
        for _, r := range sides[n] {
            ss = append(ss, fmt.Sprintf("%d%d", r[0], r[1]))
        }
        sort.Strings(ss)
        got := strings.Join(ss, " ")
        mark := "ok"
        if got ≠ pipPattern[len(ss)] {
            mark = "NOT THE USUAL PATTERN"
        }
        fmt.Printf("   side %v: %d pips at %s  %s\n", n, len(ss), got, mark)
    }
```

This code is used in section 28.

31. ⟨ Report the opposite sums and the handedness 31 ⟩ ≡

```
for _, a := range []vec {{1,0,0},{0,1,0},{0,0,1} }{
    b := vec{-a.x,-a.y,-a.z}
    fmt.Printf("␣␣␣%v␣and␣%v␣add␣to␣%d\n", a, b, len(sides[a])+len(sides[b]))
}
var n [4]vec
for _, d := range dirs {
    if k := len(sides[d]); k ≥ 1 ∧ k ≤ 3 {
        n[k] = d
    }
}
var m mat
m[0], m[1], m[2] = n[1], n[2], n[3]
fmt.Printf("␣␣␣1,␣2␣and␣3␣meet␣with␣determinant␣%d\n", det3(m))
```

This code is used in section 28.

32. "For simplicity, we'll ignore alternative setups; there are 16 ways to put spots on dice, not just two." That is worth checking too. A die assigns the numbers to the sides with opposite ones adding to seven, and then each side's pips have to be laid out: the 2, the 3 and the 6 have two orientations apiece and the others only one. Counting all of that up to the 24 rotations should give sixteen.

⟨ Functions 5 ⟩ +≡

```
var pipLayout = map[int][][][2]int{
    1: {{{3,3}}},
    2: {{{1,5},{5,1}},{{1,1},{5,5}}},
    3: {{{1,5},{3,3},{5,1}},{{1,1},{3,3},{5,5}}},
    4: {{{1,1},{1,5},{5,1},{5,5}}},
    5: {{{1,1},{1,5},{3,3},{5,1},{5,5}}},
    6: {{{1,1},{1,3},{1,5},{5,1},{5,3},{5,5}},
        {{1,1},{3,1},{5,1},{1,5},{3,5},{5,5}}},
}
func spotAt(n vec, u, v int) string {
    switch {
    case n.x ≠ 0:
        return fmt.Sprintf("%d%d%d", 3+3*n.x, u, v)
    case n.y ≠ 0:
        return fmt.Sprintf("%d%d%d", u, 3+3*n.y, v)
    }
    return fmt.Sprintf("%d%d%d", u, v, 3+3*n.z)
}
```

33. ⟨ Functions 5 ⟩ +≡

```
func spottedDice()(int, map[int]int) {
    var all []map[string]bool
    for _, a := range allMats(false) {
        ⟨ Put the numbers on the sides and the pips on the numbers 34 ⟩
    }
    cl := map[string]int{}
    stab := map[int]int{}
    for _, d := range all {
        ⟨ Reduce one spotted die by the rotations 35 ⟩
    }
    return len(cl), stab
}
```

34. Running the 48 symmetries over one fixed die gives each of the 48 ways of putting the numbers on the sides exactly once.

⟨ Put the numbers on the sides and the pips on the numbers 34 ⟩ ≡

```
side := map[int]vec{}
for i, n := range []vec {{0, 0, −1}, {0, −1, 0}, {−1, 0, 0}, {1, 0, 0}, {0, 1, 0}, {0, 0, 1} }{
    side[i + 1] = mul(a, n)
}
for i := 0; i < 8; i++ {
    d := map[string]bool{}
    for k := 1; k ≤ 6; k++ {
        pick := 0
        switch k {
        case 2:
            pick = i & 1
        case 3:
            pick = i ≫ 1 & 1
        case 6:
            pick = i ≫ 2 & 1
        }
        for _, uv := range pipLayout[k][pick] {
            d[spotAt(side[k], uv[0], uv[1])] = true
        }
    }
    all = append(all, d)
}
```

This code is used in section 33.

35. ⟨ Reduce one spotted die by the rotations 35 ⟩ ≡

```
best, fix := "", 0
for _, m := range allMats(true) {
    k := dieKey(turnDie(d, m))
    if best ≡ "" ∨ k < best {
        best = k
    }
    if k ≡ dieKey(d) {
        fix++
    }
}
cl[best]++
stab[fix]++
```

This code is used in section 33.

36. ⟨ Functions 5 ⟩ +≡

```
func dieKey(d map[string]bool) string {
    var s []string
    for k := range d {
        s = append(s, k)
    }
    sort.Strings(s)
    return strings.Join(s, "␣")
}
func turnDie(d map[string]bool, m mat) map[string]bool {
    out := map[string]bool{}
    for k := range d {
        p := vec{int(k[0] - '0'), int(k[1] - '0'), int(k[2] - '0')}
        q := add(mul(m, vec{p.x - 3, p.y - 3, p.z - 3}), vec{3, 3, 3})
        out[fmt.Sprintf("%d%d%d", q.x, q.y, q.z)] = true
    }
    return out
}
```

37. ⟨ Check the two spot patterns 37 ⟩ ≡

```
checkDie(redSpots, "red")
checkDie(greenSpots(), "green")
n, stab := spottedDice()
fmt.Printf("%d␣ways␣to␣put␣spots␣on␣a␣die,␣stabilizer␣orders␣%v\n", n, stab)
```

This code is used in section 3.

38. Colouring a red solution. For each placement I want to know which of the piece's fourteen faces show on the outside of the cube, and where.

⟨ Types 4 ⟩ +≡

```
type pinfo struct {
    pl place
    visible []bool
    name []string
    cellkey string
}
```

39. ⟨ Functions 5 ⟩ +≡

```
func info(ps []place) []pinfo {
    var out []pinfo
    for _, p := range ps {
        q := pinfo{pl: p, cellkey: cellKey(p.cells)}
        for i := range p.at {
            v := outside(add(p.at[i], p.dir[i]))
            q.visible = append(q.visible, v)
            if v {
                q.name = append(q.name, faceName(p.at[i], p.dir[i]))
            } else {
                q.name = append(q.name, "")
            }
        }
        out = append(out, q)
    }
    return out
}
```

40. A packing names sets of cells, and to paint one I need an actual placement for each set. Either of the two that fill it will do: they differ by the piece's own half-turn, so the piece they describe is the same object seen in a different frame.

⟨ Functions 5 ⟩ +≡

```
func byCells(all []pinfo) map[string]pinfo {
    m := map[string]pinfo{}
    for _, q := range all {
        if _, ok := m[q.cellkey]; ¬ok {
            m[q.cellkey] = q
        }
    }
    return m
}
```

41. Painting the red assembly says, of each piece, which of its faces are red and which are blank. A face that shows and lies on a red spot is red; a face that shows anywhere else must be blank, since the first goal allows no other spot to be seen. What is left is free.

⟨ Types 4 ⟩ +≡

```
type marks struct { red, blank [9][14]bool }
```

42. ⟨ Functions 5 ⟩ +≡

```
func markUp(p packing, one map[string]pinfo, spots map[string]bool) marks {
    var m marks
    for j, t := range p {
        q := one[t]
        for i := range q.visible {
            if ¬q.visible[i] {
                continue
            }
            if spots[q.name[i]] {
                m.red[j][i] = true
            } else {
                m.blank[j][i] = true
            }
        }
    }
    return m
}
```

43. ⟨ Count what an assembly fixes 43 ⟩ ≡

```
all := info(placements())
one := byCells(all)
ps := allPackings()
⟨ Ask how many faces a piece shows 44 ⟩
⟨ Ask whether an inner face ever shows 45 ⟩
```

This code is used in section 3.

44. ⟨ Ask how many faces a piece shows 44 ⟩ ≡

```
lo, hi, total := 99, 0, 0
for _, p := range ps {
    for _, t := range p {
        n := 0
        for _, v := range one[t].visible {
            if v {
                n++
            }
        }
        if n < lo {
            lo = n
        }
        if n > hi {
            hi = n
        }
        total += n
    }
}
fmt.Printf("a␣piece␣shows␣from␣%d␣to␣%d␣of␣its␣faces;␣%d␣faces␣to␣a␣packing\n",
    lo, hi, total/len(ps))
```

This code is used in section 43.

45. ⟨ Ask whether an inner face ever shows 45 ⟩ ≡

```
bad := 0
for _, q := range all {
    for i, f := range pieceFaces() {
        if f.inner ∧ q.visible[i] {
            bad++
        }
    }
}
fmt.Printf("%d␣faces␣to␣a␣piece,␣of␣which␣%d␣are␣inner;␣inner␣faces␣ever␣visible:␣%d\n",
    len(pieceFaces()), 2, bad)
```

This code is used in section 43.

46. Rearranging into green. Given the marks a red solution leaves, a green assembly must put every piece somewhere that shows no red face, and must cover each green spot with a face that is still free—a face already painted blank cannot be given a spot now.

```
⟨ Functions 5 ⟩ +≡
    func allowed(m marks, j int, all []pinfo, green map[string]bool) []int {
        var out []int
        for k, q := range all {
            ok := true
            for i := range q.visible {
                if ¬q.visible[i] {
                    continue
                }
                if m.red[j][i] ∨ (green[q.name[i]] ∧ m.blank[j][i]) {
                    ok = false
                    break
                }
            }
            if ok {
                out = append(out, k)
            }
        }
        return out
    }
```

47. Now the pieces are distinguishable, so the problem has nine piece items as well as the 27 cells. A secondary item per placement rides along, so that the solution says which placement each piece took; no two pieces can want the same one, since they would collide in the cells.

```
⟨ Functions 5 ⟩ +≡
    func greenProblem(m marks, all []pinfo, green map[string]bool) string {
        var b strings.Builder
        for j := 0; j < 9; j++ {
            fmt.Fprintf(&b, "p%d␣", j)
        }
        ⟨ Write the cell items 20 ⟩
        ⟨ Turn the last line into an item line with the placement tags 48 ⟩
        for j := 0; j < 9; j++ {
            ⟨ Write the options for piece j 49 ⟩
        }
        return b.String()
    }
```

48. ⟨ Turn the last line into an item line with the placement tags 48 ⟩ ≡

```
    head, rest, _ := strings.Cut(b.String(), "\n")
    head += "|"
    for k := range all {
        head += fmt.Sprintf("␣f%d", k)
    }
    b.Reset()
    b.WriteString(head + "\n" + rest)
```

This code is used in section 47.

49. ⟨ Write the options for piece j 49 ⟩ ≡

```
for _, k := range allowed(m, j, all, green) {
    var cs []string
    for _, c := range all[k].pl.cells {
        cs = append(cs, fmt.Sprintf("c%d%d%d", c.x, c.y, c.z))
    }
    sort.Strings(cs)
    fmt.Fprintf(&b, "p%d␣%s␣f%d\n", j, strings.Join(cs, "␣"), k)
}
```

This code is used in section 47.

50. ⟨ Functions 5 ⟩ +≡

```
func countGreen(m marks, all []pinfo, green map[string]bool)(int, [][9]int) {
    xc := cells.NewXCC()
    n := 0
    var got [][9]int
    in := greenProblem(m, all, green)
    for sol := range xc.Dance(strings.NewReader(in)).Solutions {
        n++
        var g [9]int
        for _, o := range sol {
            var j, k int
            fmt.Sscanf(o[0], "p%d", &j)
            fmt.Sscanf(o[len(o) - 1], "f%d", &k)
            g[j] = k
        }
        got = append(got, g)
    }
    return n, got
}
```

51. A red solution and a green one together specify 54 faces each. A face specified by both must be blank in both—a red face is hidden in the green assembly, and a green spot has to go on a face the red assembly left free—so the number of faces left unspecified is exactly the number specified twice.

⟨ Rearrange every red solution 51 ⟩ ≡

```
all := info(placements())
one := byCells(all)
red := map[string]bool{}
for _, s := range redSpots {
    red[s] = true
}
green := map[string]bool{}
for _, s := range greenSpots() {
    green[s] = true
}
ps := allPackings()
⟨ Try to rearrange each red solution in turn 52 ⟩
```

This code is used in section 3.

52. ⟨ Try to rearrange each red solution in turn 52 ⟩ ≡

```
nred, pairs, best, bestAt := 0, 0, 0, 0
unspec := map[int]int{}
for idx := *lo; idx < *hi ∧ idx < len(ps); idx++ {
    m := markUp(ps[idx], one, red)
    n, got := countGreen(m, all, green)
    if n ≡ 0 {
        continue
    }
    nred++
    pairs += n
    if n > best {
        best, bestAt = n, idx
    }
    ⟨ Count the faces each pair leaves unspecified 53 ⟩
}
fmt.Printf("%d␣of␣%d␣red␣solutions␣can␣be␣rearranged\n", nred, *hi − *lo)
fmt.Printf("␣␣%d␣red+green␣pairs;␣most␣greens␣%d,␣from␣red␣solution␣%d\n",
    pairs, best, bestAt)
fmt.Printf("␣␣␣faces␣left␣unspecified:␣%v\n", unspec)
```

This code is used in section 51.

53. ⟨ Count the faces each pair leaves unspecified 53 ⟩ ≡

```
for _, g := range got {
    k := 0
    for j := 0; j < 9; j++ {
        q := one[ps[idx][j]]
        r := all[g[j]]
        for i := range q.visible {
            if q.visible[i] ∧ r.visible[i] {
                k++
            }
        }
    }
    unspec[k]++
}
```

This code is used in section 52.

Twice Dice